МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Алгоритмы и структуры данных»

Студент гр. 4354 _____ Фанасюткин Р.В.

Преподаватель _____ Пестерев Д.О.

Санкт-Петербург

2026

Теоритическая часть:

Алгоритм RLE (англ. Run-Length Encoding) -алгоритм сжатия, заменяющий идущие подряд одинаковые символы парой (повторяющийся символ, количество повторений). Например, строчку aaababbcbbb он переводит в (a, 3) (b, 1) (a, 1) (b, 2) (c, 1) (b, 3). Этот алгоритм эффективен для строк, содержащих много цепочек повторяющихся символов.

Кодирование и декодирование за O(n), память O(n).

Преобразование MTF (англ. move-to-front, движение к началу) -алгоритм кодирования, используемый для предварительной обработки данных (обычно потока байтов) перед сжатием, разработанный для улучшения эффективности последующего кодирования.

Алгоритм декодирования:

- Выполняется проход по закодированным данным.
- Если встречается число, оно интерпретируется как количество повторений символа.
- Следующий символ записывается в выходной поток указанное число раз.

Пространственная сложность:

В лучшем случае, когда строка состоит из одной серии повторяющихся символов, требуется $O(1)$ дополнительной памяти.

В худшем случае, когда повторяющихся символов нет, каждый символ кодируется отдельно, поэтому размер выходных данных составляет $O(n)$.

Символ	**Список**	**Вывод**
B	ABC	1
C	BAC	2
A	CBA	2
B	ACB	2
A	BAC	1
A	ABC	0
A	ABC	0

Таблица 1: Результаты обработки символов

Кодирование, декодирование и память O(n).

Алгоритм Хаффмана (англ. Huffman's algorithm) -алгоритм оптимального префиксного кодирования алфавита.

Пусть $A = \{a_1, a_2, \ldots, a_n\}$ -алфавит из n различных символов,

$W = \{w_1, w_2, \ldots, w_n\}$ -соответствующий ему набор положительных целых весов.

Тогда набор бинарных кодов $C = \{c_1, c_2, \ldots, c_n\}$, где c_i является кодом для символа a_i, такой, что:

- c_i не является префиксом для c_j, при $i \neq j$,
- сумма

$$\sum_{i=1}^{n} w_i \cdot |c_i|$$

минимальна, где $|c_i|$ - длина кода c_i.

Время: O(m + n log n), память: O(k + m).

Алгоритм декодирования:

- Закодированный поток считывается побитово.
- Выполняется обход дерева Хаффмана:
  - переход влево соответствует биту 0,
  - переход вправо соответствует биту 1.
- При достижении листа дерева соответствующий символ записывается в выходной поток.
- После этого обход начинается снова от корня дерева.

Декодирование является однозначным, так как используются префиксные коды.

Пример: Закодируем слово abracadabra

Узел : a b r c d

Вес : 5 2 2 1 1

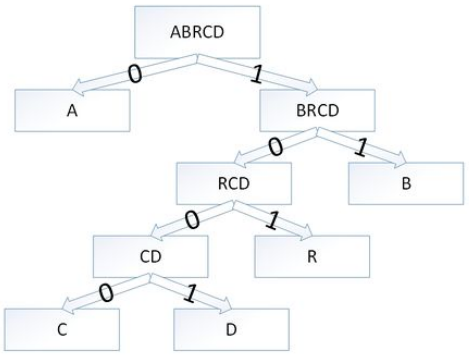


Рис. 1 – Дерево кодов Хафмана

Арифметическое кодирование (англ. Arithmetic coding) -алгоритм сжатия информации без потерь, который при кодировании ставит в соответствие тексту вещественное число из отрезка [0;1). Данный метод, как и алгоритм Хаффмана, является энтропийным, то есть длина кода конкретного символа зависит от частоты встречаемости этого символа в тексте. Арифметическое кодирование показывает более высокие результаты сжатия, чем алгоритм Хаффмана, для данных с неравномерными распределениями вероятностей кодируемых символов.

Временная сложность:

Кодирование и декодирование выполняются за O(n), где n -длина входных данных.

Пространственная сложность:

Для хранения вероятностей символов и текущего интервала требуется O(k) памяти, где k -размер алфавита.

Пример кодирования:

Символ	Частота появления
a	0.571429
b	0.285714
c	0.142857

Таблица 2: Частоты символов

Символ	Левая	Правая
	0	1
a	0	0.571429
b	0.326531	0.489796
a	0.326531	0.419825
c	0.406497	0.419825
a	0.406497	0.414113
b	0.410849	0.413025
a	0.410849	0.412093

Таблица 3: Интервалы

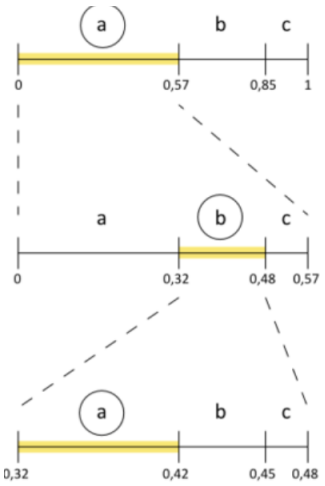


Таблица 4: Иллюстрация

Пример декодирования:

Декодируемый символ	Код
a	0.411471
b	0.720074
a	0.520259
c	0.910454
a	0.373178
b	0.653061
a	0.285714

Таблица 5: Результаты декодирования

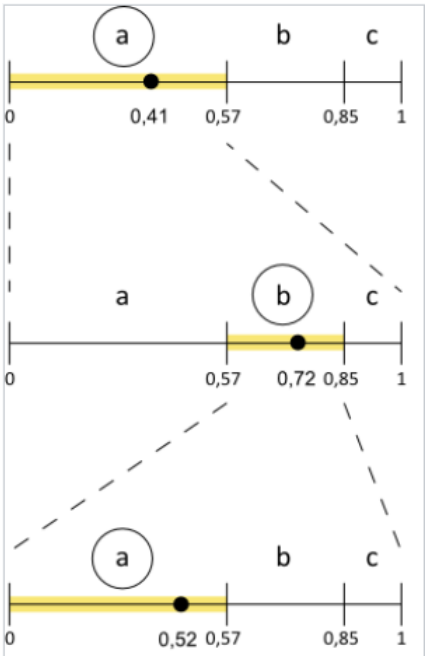


Таблица 6: Иллюстрация процесса

Преобразование Барроуза -Уилера (англ. Burrows-Wheeler transform) -алгоритм, используемый для предварительной обработки данных перед сжатием, разработанный для улучшения эффективности последующего кодирования. Преобразование Барроуза -Уилера меняет порядок символов во входной строке таким об-

разом, что повторяющиеся подстроки образуют на выходе идущие подряд последовательности одинаковых символов.

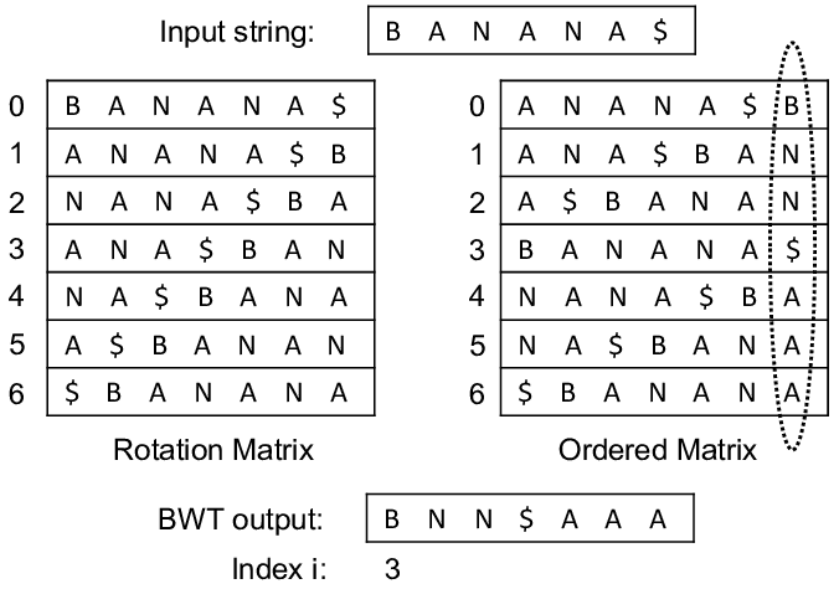


Рис. 2

Обратное преобразование:

- Символы последнего столбца сортируются, формируя первый столбец таблицы.
- Между первым и последним столбцами устанавливается соответствие.
- Постепенно восстанавливаются строки таблицы циклических сдвигов.
- По сохранённому индексу определяется исходная строка.

LZ77:

Алгоритм сжатия со «скользящим окном»: есть буфер поиска (уже обработанные данные) и буфер просмотра (вход).

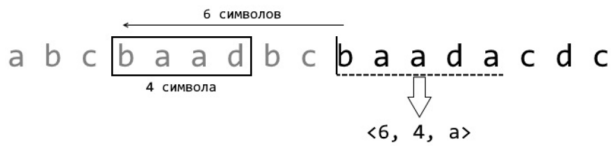


Рис. 3

Алгоритм кодирования:

- Ищется максимальное совпадение последовательности из буфера просмотра в буфере поиска.
- Формируется тройка (смещение, длина, символ), где: смещение -расстояние до начала совпадения, длина -длина совпадения, символ -следующий символ после совпадения.
- Если совпадения нет -(0, 0, символ).
- Окно сдвигается на (длина + 1).

Алгоритм декодирования:

- Читаются тройки (смещение, длина, символ).
- Если смещение = 0 -> символ записывается в выход.
- Если смещение > 0 -> копируется фрагмент из уже декодированных данных и добавляется символ.

Сложность: O(mkn), декодирование O(n). Память: O(m + k + n).

LZSS:

Алгоритм является модификацией LZ77. Основное отличие заключается в том, что ссылки на повторяющиеся фрагменты кодируются только при достижении минимальной длины совпадения.

Алгоритм кодирования:

- Аналогично LZ77 ищется максимальное совпадение в буфере поиска.
- Если длина совпадения $\geq$ min_len, формируется ссылка: (флаг, смещение, длина).
- Если совпадение отсутствует или его длина меньше min_len, формируется литерал: (флаг, символ).
- Окно сдвигается на длину совпадения или на 1 (для литерала).
- Использование флагов позволяет различать ссылки и литералы при декодировании.

Алгоритм декодирования:

- Читается флаг из входного потока.
- Если флаг указывает на литерал -> символ записывается в выход.
- Если флаг указывает на ссылку -> читаются смещение и длина.
- Из уже восстановленных данных копируется фрагмент и добавляется в выходной поток.

Сложность: O(mkn), декодирование O(n). Память: O(m + k + n).

LZ78:

Алгоритм использует глобальный словарь, который формируется динамически в процессе обработки входных данных. В отличие от LZ77, скользящее окно не используется.

Словарь формируется динамически в процессе кодирования и одинаково восстанавливается при декодировании, поэтому передача словаря отдельно не требуется.

Алгоритм кодирования:

- Последовательно обрабатывается входной поток и ищется самая длинная подстрока, уже присутствующая в словаре.
- Если совпадение найдено, формируется пара (индекс, символ), где индекс - позиция строки в словаре, а символ -следующий символ после найденной подстроки.
- Если совпадение не найдено, формируется пара (0, символ).
- Новая строка (совпадение + символ) добавляется в словарь.

Алгоритм декодирования:

- Словарь инициализируется пустой строкой с индексом 0.
- Для каждой пары (индекс, символ): извлекается строка по индексу.
- К строке добавляется символ, формируется новая строка.

- Новая строка записывается в выходной поток и добавляется в словарь.

Сложность: кодирование до $O(n^2)$,

декодирование $O(m)$.

Память: $O(k + n)$.

LZSS (Lempel-Ziv-Storer-Szymanski) -это модификация алгоритма LZ77, в которой используется идея сокращения избыточных ссылок на короткие совпадения.

Основное отличие от LZ77 заключается в том, что вместо всегда использования троек *(смещение, длина, символ)* вводится порог минимальной длины совпадения.

Алгоритм кодирования:

СЛОВАРЬ(8)	БУФЕР(5)	КОД	ДЛИНА КОДА
"........"	"КРАСН"	0'К'	9
".......К"	"РАСНА"	0'Р'	9
"......КР"	"АСНАЯ"	0'А'	9
".....КРА"	"СНАЯ "	0'С'	9
"....КРАС"	"НАЯ К"	0'Н'	9
"...КРАСН"	"АЯ КР"	1<5,1>	7
"..КРАСНА"	"Я КРА"	0'Я'	9
".КРАСНАЯ"	"КРАС"	0' '	9
"КРАСНАЯ "	"КРАСК"	1<0,4>	7
"НАЯ КРАС"	"КА..."	1<4,1>	7
"АЯ КРАСК"	"А...."	1<0,1>	7

Рис. 4

Алгоритм декодирования:

$0'К'0'Р'0'А'0'С'0'Н'1\langle 5,1\rangle 0'Я'0''1\langle 0,4\rangle 1\langle 4,1\rangle 1\langle 0,1\rangle$

ВХОДНОЙ КОД	ПЕЧАТЬ	СЛОВАРЬ
0 'К'	"К"	".......К"
0 'Р'	"Р"	"......КР"
0 'А'	"А"	".....КРА"
0 'С'	"С"	"....КРАС"
0 'Н'	"Н"	"...КРАСН"
1 <5,1>	"А"	"..КРАСНА"
0 'Я'	"Я"	".КРАСНАЯ"
0 ' '		"КРАСНАЯ "
1 <0,4>	"КРАС"	"НАЯ КРАС"
1 <4,1>	"К"	"АЯ КРАСК"
1 <0,1>	"А"	"Я КРАСКА"

Рис. 5

Пространственная сложность: $O(m+k+n)$, где m -размер окна, k -буфер, n -входные данные.

LZW (Lempel-Ziv-Welch) -словарный алгоритм сжатия без потерь, основанный на динамическом построении словаря подстрок. В отличие от LZ78, он не передаёт символы вместе с индексами, а использует только индексы уже построенного словаря.

Алгоритм кодирования:

- Инициализируется словарь всеми одиночными символами алфавита.
- Читается входная строка и формируется текущая последовательность w.
- Для каждого следующего символа c:
  - если $w + c$ есть в словаре, $w := w + c$;
  - иначе:
    - выводится код w;
    - добавляется $w + c$ в словарь;
    - $w := c$;
- После завершения работы выводится код последней последовательности w.

Алгоритм декодирования:

- Инициализируется словарь всеми одиночными символами.
- Читается первый код и записывается как текущая строка w.
- Для каждого следующего кода k:
  - если k есть в словаре -извлекается строка $entry$;
  - если k отсутствует (особый случай) -$entry = w + w[0]$;
  - выводится $entry$;
  - в словарь добавляется $w + entry[0]$;
  - $w := entry$.

Временная сложность:

- Кодирование: $O(n)$ при эффективной реализации словаря (хеш-таблица).
- Декодирование: $O(n)$.

Пространственная сложность:

- $O(k)$, где k -количество элементов словаря (растёт с длиной входа).

Особенность: LZW не требует передачи словаря, так как он полностью восстанавливается на стороне декодера.

Практическая часть:

Подготовим тестовый набор из файлов и картинок.



Рис. 6 – Черно-белое изображение



Рис. 7 – Изображение в оттенках серого



Рис. 8 – Цветное изображение

Переведём изображения в собственный raw формат, в котором один/три байта задают значение цветности пикселя. Дополним файл метаданными о типе (чб/оттенки серого/цветное)

Ссылка на репозиторий: https://github.com/romanfan30/algo_lab_1

Обработка результатов:

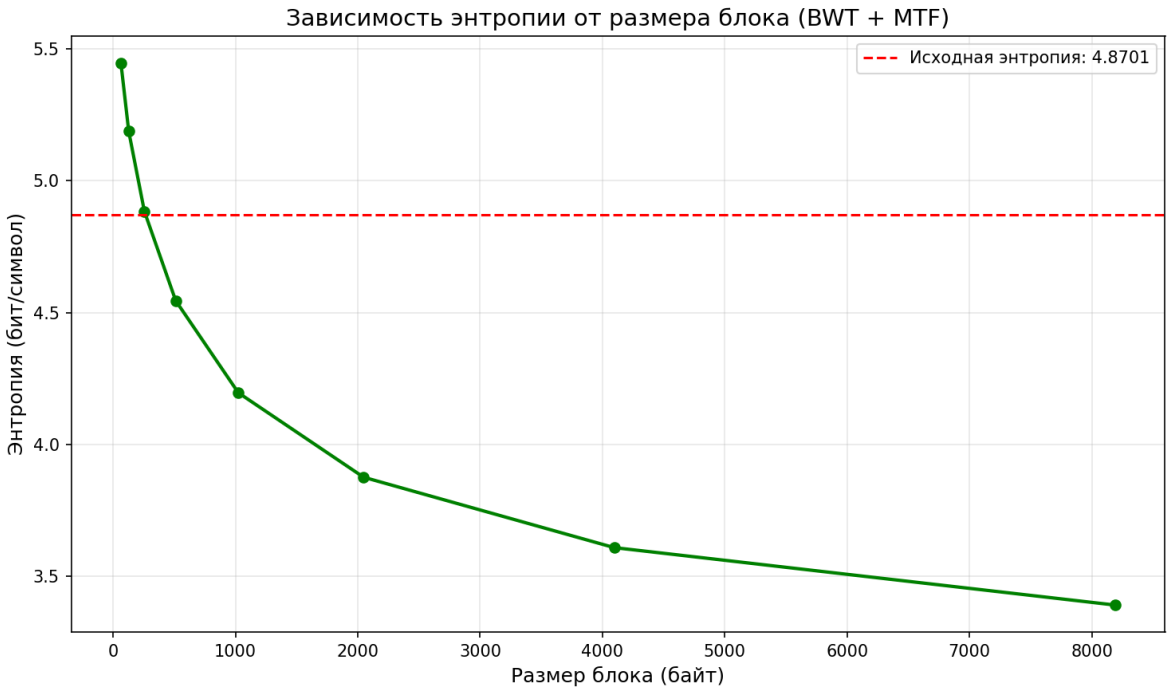


Рис. 9 – Построение DAG Compression

В ходе исследования было установлено, что с увеличением размера блока энтропия уменьшается, поскольку структура данных становится более упорядоченной за счёт появления повторяющихся символов. Однако слишком большие блоки увеличивают время обработки без заметного улучшения сжатия.

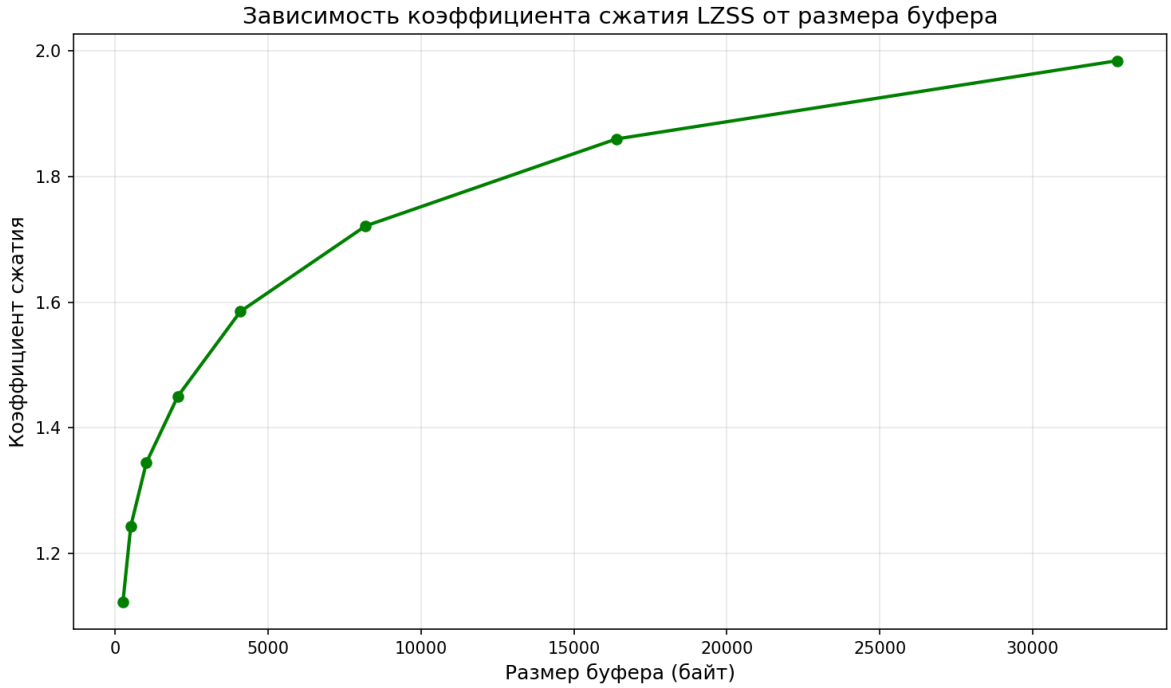


Рис. 10 – Зависимость энтропии от размера блока(BWT + MTF)

При увеличении размера буфера коэффициент сжатия возрастает, но буфер более 10000 байт почти не влияет на результат, при этом замедляя обработку.

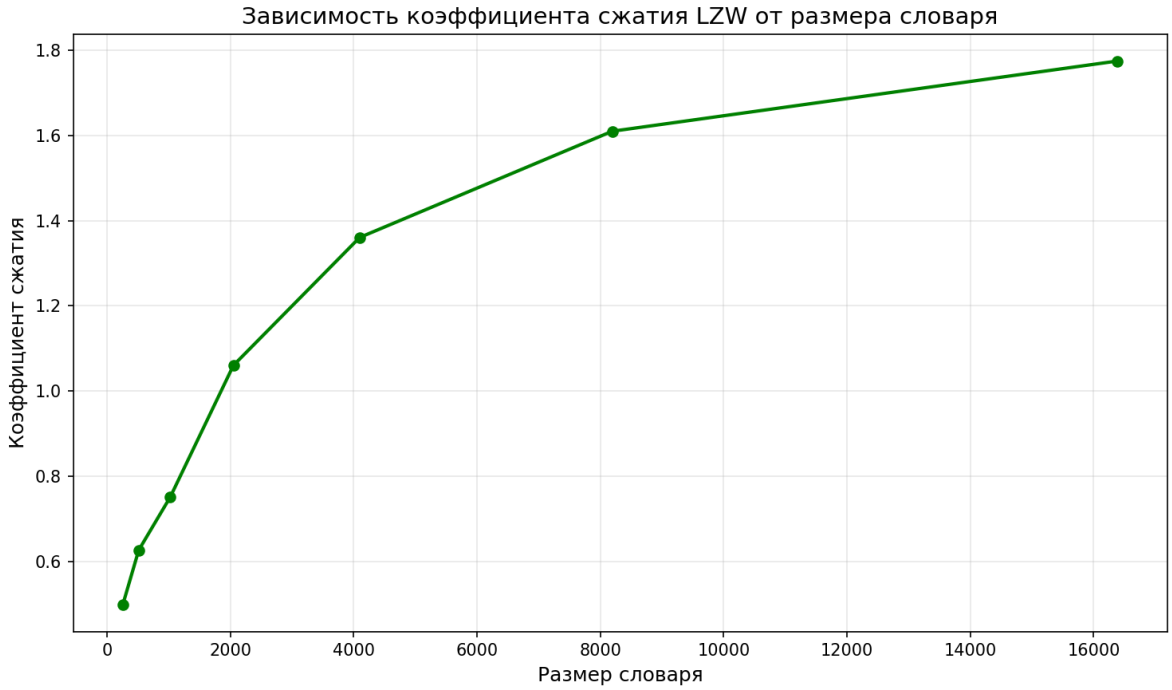


Рис. 11 – Зависимость коэффицента сжатия LZW от размера словаря

Рост размера словаря также повышает коэффициент сжатия, однако после 4000 байт изменения становятся незначительными. Оптимальным размером словаря является 4096 байт.

Компрессор	**Исходный размер**	**Сжатый размер**	**Коэффициент**
HA	937.51 KB	160.65 KB	5.848
RLE		65.71 KB	14.286
BWT + RLE		97.28 KB	9.615
BWT + MTF + HA		124.15 KB	7.576
BWT + MTF + RLE + HA		44.91 KB	20.833
LZSS		36.12 KB	25.641
LZSS + HA		28.99 KB	32.258
LZW		35.76 KB	26.316
LZW + HA		29.91 KB	31.250

Таблица 7: Результаты сжатия черно-белого изображения

Компрессор	**Исходный размер**	**Сжатый размер**	**Коэффициент**
HA	937.51 KB	902.75 KB	1.040
RLE		874.43 KB	1.072
BWT + RLE		886.43 KB	1.058
BWT + MTF + HA		575.09 KB	1.631
BWT + MTF + RLE + HA		590.98 KB	1.585
LZSS		854.81 KB	1.096
LZSS + HA		772.55 KB	1.214
LZW		1.25 MB	0.735
LZW + HA		945.01 KB	0.992

Таблица 8: Результаты сжатия изображения в оттенках серого

Компрессор	**Исходный размер**	**Сжатый размер**	**Коэффициент**
HA	2.75 MB	2.70 MB	1.017
RLE		2.74 MB	1.002
BWT + RLE		2.62 MB	1.048
BWT + MTF + HA		2.28 MB	1.205
BWT + MTF + RLE + HA		2.30 MB	1.195
LZSS		2.57 MB	1.067
LZSS + HA		2.35 MB	1.168
LZW		4.77 MB	0.576
LZW + HA		3.18 MB	0.864

Таблица 9: Результаты сжатия файла color.raw

Компрессор	**Исходный размер**	**Сжатый размер**	**Коэффициент**
HA	200.00 KB	103.15 KB	1.938
RLE		199.79 KB	1.001
BWT + RLE		105.02 KB	1.905
BWT + MTF + HA		73.08 KB	2.740
BWT + MTF + RLE + HA		64.58 KB	3.096
LZSS		110.04 KB	1.818
LZSS + HA		86.18 KB	2.320
LZW		99.34 KB	2.012
LZW + HA		84.37 KB	2.370

Таблица 10: Результаты сжатия текстового файла Haksli_200kb.txt

Компрессор	**Исходный размер**	**Сжатый размер**	**Коэффициент**
HA	1.10 MB	1017.17 KB	1.107
RLE		1.06 MB	1.035
BWT + RLE		873.00 KB	1.290
BWT + MTF + HA		723.70 KB	1.555
BWT + MTF + RLE + HA		742.26 KB	1.517
LZSS		840.26 KB	1.340
LZSS + HA		777.09 KB	1.449
LZW		1.68 MB	0.657
LZW + HA		1.14 MB	0.968

Таблица 11: Результаты сжатия файла part_1_1mb.exe

Компрессор	Исходный размер	Сжатый размер	Коэффициент
HA	9.54 MB	6.12 MB	1.560
RLE		9.49 MB	1.005
BWT + RLE		7.39 MB	1.290
BWT + MTF + HA		4.38 MB	2.179
BWT + MTF + RLE + HA		4.68 MB	2.037
LZSS		6.16 MB	1.548
LZSS + HA		5.12 MB	1.862
LZW		7.70 MB	1.238
LZW + HA		6.44 MB	1.481

Таблица 12: Результаты сжатия файла enwik7

Код:

Задание 1:

image_converter_.py:

```python
"""
конвертация изображений в RAW формат
задание 1, пункт 1
"""

import numpy as np
from PIL import Image
import struct


def convert_jpeg_to_raw(input_path, output_path):
    """конвертация JPEG в RAW формат с метаданными"""
    img = Image.open(input_path)

    if img.mode == '1' or img.mode == 'P':
        img = img.convert('1')
        img_type = 1  # черно-белое
    elif img.mode == 'L':
        img_type = 2  # оттенки серого
    else:
        img = img.convert('RGB')
        img_type = 3  # цветное

    width, height = img.size
```

```
    raw_data = np.array(img)

    # заголовок: тип (1 байт), ширина (4), высота (4)
    with open(output_path, 'wb') as f:
        f.write(struct.pack('B', img_type))
        f.write(struct.pack('I', width))
        f.write(struct.pack('I', height))
        raw_data.tofile(f)

    print(f"сохранено: {output_path}")
    print(f"тип: {img_type}, размер: {width}x{height}")

if __name__ == "__main__":
    convert_jpeg_to_raw('colors.png', '../analysis/color.raw')
```

rle.py:

```
"""
RLE (Run-Length Encoding) - кодирование длин серий
задание 1, пункт 2
"""

def rle_encode(data):
    """базовое RLE кодирование с 8-битными кодами"""
    if not data:
        return b''

    result = bytearray()
    i = 0
    n = len(data)

    while i < n:
        run_len = 1
        while (i + run_len < n) and (data[i + run_len] == data[i]) and (run_len < 127):
            run_len += 1

        if run_len >= 3:
            result.append(run_len)
            result.append(data[i])
            i += run_len
        else:
            start = i
```

```
            while i < n:
                test_len = 1
                while (i + test_len < n) and (data[i + test_len] == data[i]) and (test_len <
    127):
                    test_len += 1

                if test_len >= 3:
                    break
                i += 1

            unique_len = i - start
            if unique_len > 0:
                if unique_len > 127:
                    unique_len = 127
                ctrl = 0x80 | unique_len
                result.append(ctrl)
                result.extend(data[start:i])

    return bytes(result)


def rle_decode(data):
    """базовое RLE декодирование"""
    if not data:
        return b''

    result = bytearray()
    i = 0
    n = len(data)

    while i < n:
        ctrl = data[i]
        i += 1
        length = ctrl & 0x7F

        if (ctrl & 0x80) == 0:
            if i >= n:
                break
            symbol = data[i]
            i += 1
            result.extend([symbol] * length)
        else:
```

```
            if i + length > n:
                break
            result.extend(data[i:i + length])
            i += length

    return bytes(result)


def rle_encode_with_len(data, ms=8, mc=8):
    """
    RLE кодирование с настраиваемой длиной символа и управляющего кода
    ms - длина кода символа в битах
    mc - длина управляющего символа в битах
    """
    if not data:
        return b''

    def bytes_to_symbols(data_bytes, ms_val):
        symbol_bytes = ms_val // 8
        symbols = []
        for idx in range(0, len(data_bytes), symbol_bytes):
            chunk = data_bytes[idx:idx + symbol_bytes]
            if len(chunk) < symbol_bytes:
                chunk = chunk + b'\x00' * (symbol_bytes - len(chunk))
            value = 0
            for j, b in enumerate(chunk):
                value |= (b << (8 * j))
            symbols.append(value)
        return symbols

    def symbols_to_bytes(symbols_list, ms_val):
        symbol_bytes = ms_val // 8
        result = bytearray()
        for sym in symbols_list:
            for i in range(symbol_bytes):
                result.append((sym >> (8 * i)) & 0xFF)
        return bytes(result)

    symbols = bytes_to_symbols(data, ms)
    result = bytearray()
    i = 0
    n = len(symbols)
```

```
    max_count = (1 << (mc - 1)) - 1

    while i < n:
        run_len = 1
        while (i + run_len < n) and (symbols[i + run_len] == symbols[i]) and (run_len < max_count):
            run_len += 1

        if run_len >= 2:
            result.append(run_len)
            sym_bytes = symbols_to_bytes([symbols[i]], ms)
            result.extend(sym_bytes)
            i += run_len
        else:
            start = i
            while i < n:
                test_len = 1
                while (i + test_len < n) and (symbols[i + test_len] == symbols[i]) and (test_len < max_count):
                    test_len += 1
                if test_len >= 3:
                    break
                i += 1

            unique_len = i - start
            if unique_len > 0:
                result.append(0x80 | unique_len)
                for k in range(start, i):
                    sym_bytes = symbols_to_bytes([symbols[k]], ms)
                    result.extend(sym_bytes)

    return bytes(result)


def rle_decode_with_len(data, ms=8, mc=8):
    """RLE декодирование с настраиваемой длиной символа"""
    if not data:
        return b''

    def symbols_to_bytes(symbols_list, ms_val):
        symbol_bytes = ms_val // 8
        result = bytearray()
```

```
        for sym in symbols_list:
            for i in range(symbol_bytes):
                result.append((sym >> (8 * i)) & 0xFF)
        return bytes(result)

    result_symbols = []
    i = 0
    n = len(data)
    symbol_bytes = ms // 8

    while i < n:
        ctrl = data[i]
        i += 1
        length = ctrl & 0x7F

        if (ctrl & 0x80) == 0:
            if i + symbol_bytes > n:
                break
            sym_bytes = data[i:i + symbol_bytes]
            i += symbol_bytes
            symbol = 0
            for j, b in enumerate(sym_bytes):
                symbol |= (b << (8 * j))
            result_symbols.extend([symbol] * length)
        else:
            for _ in range(length):
                if i + symbol_bytes > n:
                    break
                sym_bytes = data[i:i + symbol_bytes]
                i += symbol_bytes
                symbol = 0
                for j, b in enumerate(sym_bytes):
                    symbol |= (b << (8 * j))
                result_symbols.append(symbol)

    return symbols_to_bytes(result_symbols, ms)


if __name__ == "__main__":
    print("RLE с Ms и Mc")
    print("Ms=16")
    test_ms16 = bytes([0xCF, 0xCE, 0xCF, 0xCE, 0xCF, 0xCE])
```

```
    enc_ms16 = rle_encode_with_len(test_ms16, ms=16, mc=8)
    print(f"0x{' 0x'.join(f'{b:02X}' for b in test_ms16)} -> 0x{' 0x'.join(f'{b:02X}' for b in enc_ms16)}")
    dec_ms16 = rle_decode_with_len(enc_ms16, ms=16, mc=8)
    print(f"0x{' 0x'.join(f'{b:02X}' for b in enc_ms16)} -> 0x{' 0x'.join(f'{b:02X}' for b in dec_ms16)}")

    print("\nMs=24")
    test_ms24 = bytes([0xCF, 0xCE, 0xCF, 0xCF, 0xCE, 0xCF])
    enc_ms24 = rle_encode_with_len(test_ms24, ms=24, mc=8)
    print(f"0x{' 0x'.join(f'{b:02X}' for b in test_ms24)} -> 0x{' 0x'.join(f'{b:02X}' for b in enc_ms24)}")
    dec_ms24 = rle_decode_with_len(enc_ms24, ms=24, mc=8)
    print(f"0x{' 0x'.join(f'{b:02X}' for b in enc_ms24)} -> 0x{' 0x'.join(f'{b:02X}' for b in dec_ms24)}")
```

Задание 2:

bwt.py:

```
"""
преобразование барроуза-уиллера (BWT)
задание 2, пункт 2
"""


def bwt_encode_block(data):
    """BWT кодирование с построением матрицы"""
    if not data:
        return b'', 0

    n = len(data)
    matrix = []
    for i in range(n):
        row = data[i:] + data[:i]
        matrix.append(row)

    for i in range(n):
        for j in range(i + 1, n):
            if matrix[i] > matrix[j]:
                matrix[i], matrix[j] = matrix[j], matrix[i]

    last = bytearray()
```

```
    for row in matrix:
        last.append(row[-1])

    for i, row in enumerate(matrix):
        if row == data:
            idx = i
            break

    print(matrix)
    return bytes(last), idx


def bwt_decode_matrix(last, idx):
    """BWT декодирование через построение матрицы"""
    n = len(last)
    matrix = [bytearray() for _ in range(n)]

    for _ in range(n):
        for i in range(n):
            matrix[i].insert(0, last[i])
        for i in range(n):
            for j in range(i + 1, n):
                if matrix[i] > matrix[j]:
                    matrix[i], matrix[j] = matrix[j], matrix[i]

    return bytes(matrix[idx])


def bwt_decode_fast(last, idx):
    """BWT декодирование с использованием перестановки"""
    n = len(last)

    first = sorted(last)

    positions = [[] for _ in range(256)]
    for i, ch in enumerate(last):
        positions[ch].append(i)

    next_arr = [0] * n
    counts = [0] * 256
    for i, ch in enumerate(first):
        next_arr[i] = positions[ch][counts[ch]]
```

```
        counts[ch] += 1

    res = bytearray()
    curr = idx
    for _ in range(n):
        res.append(first[curr])
        curr = next_arr[curr]

    return bytes(res)


def compress_file(data, block_size=1024):
    """BWT + RLE с блочной обработкой"""
    from exercise1.rle import rle_encode

    if not data:
        return b''

    res = bytearray()
    n = len(data)

    res.append((block_size >> 0) & 0xFF)
    res.append((block_size >> 8) & 0xFF)
    res.append((block_size >> 16) & 0xFF)
    res.append((block_size >> 24) & 0xFF)

    i = 0
    while i < n:
        block = data[i:i + block_size]

        bwt_block, idx = bwt_encode_block(block)
        rle_block = rle_encode(bwt_block)

        res.append((idx >> 0) & 0xFF)
        res.append((idx >> 8) & 0xFF)
        res.append((idx >> 16) & 0xFF)
        res.append((idx >> 24) & 0xFF)

        rle_len = len(rle_block)
        res.append((rle_len >> 0) & 0xFF)
        res.append((rle_len >> 8) & 0xFF)
        res.append((rle_len >> 16) & 0xFF)
```

```
        res.append((rle_len >> 24) & 0xFF)

        res.extend(rle_block)

        i += block_size

    return bytes(res)


def decompress_file(data):
    """декодирование BWT + RLE"""
    from exercise1.rle import rle_decode

    if not data:
        return b''

    res = bytearray()
    i = 0

    block_size = (data[i] | (data[i + 1] << 8) | (data[i + 2] << 16) | (data[i + 3] << 24))
    i += 4

    while i < len(data):
        idx = (data[i] | (data[i + 1] << 8) | (data[i + 2] << 16) | (data[i + 3] << 24))
        i += 4

        rle_len = (data[i] | (data[i + 1] << 8) | (data[i + 2] << 16) | (data[i + 3] << 24))
        i += 4

        rle_block = data[i:i + rle_len]
        i += rle_len

        bwt_block = rle_decode(rle_block)
        block = bwt_decode_fast(bwt_block, idx)

        res.extend(block)

    return bytes(res)


def rle_encode_block(data):
    """RLE кодирование блока"""
```

```
    if not data:
        return b''

    res = bytearray()
    i = 0
    n = len(data)

    while i < n:
        j = i + 1
        while j < n and data[j] == data[i] and (j - i) < 127:
            j += 1

        run_len = j - i

        if run_len >= 3:
            res.append(run_len)
            res.append(data[i])
            i = j
        else:
            start = i
            while i < n and (i - start) < 127:
                k = i
                while k + 1 < n and data[k + 1] == data[k]:
                    k += 1
                if k - i + 1 >= 3:
                    break
                i += 1

            unique_len = i - start
            if unique_len > 0:
                res.append(0x80 | unique_len)
                res.extend(data[start:i])

    return bytes(res)


def rle_decode_block(data):
    """RLE декодирование блока"""
    if not data:
        return b''

    res = bytearray()
```

```
    i = 0
    n = len(data)

    while i < n:
        ctrl = data[i]
        i += 1
        length = ctrl & 0x7F

        if (ctrl & 0x80) == 0:
            if i >= n:
                break
            sym = data[i]
            i += 1
            res.extend([sym] * length)
        else:
            if i + length > n:
                break
            res.extend(data[i:i + length])
            i += length

    return bytes(res)


if __name__ == "__main__":
    test_data = bytes([0x62, 0x61, 0x6e, 0x61, 0x6e, 0x61])
    print(f"вход: {test_data.hex().upper()} ({test_data.decode()})")

    bwt_data, idx = bwt_encode_block(test_data)
    print(f"BWT: {bwt_data.hex().upper()} (индекс {idx})")

    decoded_matrix = bwt_decode_matrix(bwt_data, idx)
    print(f"обратное BWT (матрица): {decoded_matrix.hex().upper()}")

    decoded_fast = bwt_decode_fast(bwt_data, idx)
    print(f"обратное BWT (быстрое): {decoded_fast.hex().upper()}")

```

entropy.py:

```
"""
расчет энтропии сообщения
задание 2, пункт 1.1
```

```
"""

import math
import matplotlib.pyplot as plt


def calculate_entropy(data, symbol_bytes=1):
    """
    расчет энтропии сообщения
    data: байтовая строка
    symbol_bytes: длина кода символа в байтах
    """
    if not data:
        return 0

    n = len(data)
    symbols = []
    for i in range(0, n, symbol_bytes):
        chunk = data[i:i + symbol_bytes]
        if len(chunk) < symbol_bytes:
            chunk = chunk + b'\x00' * (symbol_bytes - len(chunk))
        symbols.append(chunk)

    freq = {}
    for sym in symbols:
        if sym in freq:
            freq[sym] += 1
        else:
            freq[sym] = 1
    total = len(symbols)

    entropy = 0
    for count in freq.values():
        p = count / total
        entropy -= p * math.log2(p)

    return entropy


def analyze_entropy_by_symbol_len(data, symbol_bytes_list):
    """анализ зависимости энтропии от длины символа"""
    results = {'symbol_bytes': symbol_bytes_list, 'entropy': []}
```

```
    for sb in symbol_bytes_list:
        entropy = calculate_entropy(data, sb)
        results['entropy'].append(entropy)
        print(f"длина символа: {sb} байт  энтропия: {entropy:.4f} бит/символ")

    return results


def plot_entropy_vs_symbol_len(results):
    """график зависимости энтропии от длины символа"""
    plt.figure(figsize=(10, 6))
    plt.plot(results['symbol_bytes'], results['entropy'], 'bo-', linewidth=2, markersize=8)
    plt.xlabel('длина символа (байт)', fontsize=12)
    plt.ylabel('энтропия (бит/символ)', fontsize=12)
    plt.title('зависимость энтропии от длины кода символа', fontsize=14)
    plt.grid(True, alpha=0.3)
    plt.xticks(results['symbol_bytes'])

    for i, (x, y) in enumerate(zip(results['symbol_bytes'], results['entropy'])):
        plt.annotate(f'{y:.4f}', (x, y), textcoords="offset points",
                     xytext=(0, 10), ha='center', fontsize=10)

    plt.tight_layout()
    plt.show()


def test_entropy_for_english_text():
    """тест энтропии для английского текста"""
    english_text = """

When he was nearly thirteen, my brother Jem got his arm badly broken at the elbow. When it healed, and Jems fears of never being able to play football were assuaged, he was seldom self-conscious about his injury. His left arm was somewhat shorter than his right; when he stood or walked, the back of his hand was at right angles to his body, his thumb parallel to his thigh. He couldnt have cared less, so long as he could pass and punt.

When enough years had gone by to enable us to look back on them, we sometimes discussed the events leading to his accident. I maintain that the Ewells started it all, but Jem, who was four years my senior, said it started long before that. He said it began the summer Dill came to us, when Dill first gave us the idea of making Boo Radley come out.

```

```
I said if he wanted to take a broad view of the thing, it really began with Andrew Jackson. If General Jackson hadnt run the Creeks up the creek, Simon Finch would never have paddled up the Alabama, and where would we be if he hadnt? We were far too old to settle an argument with a fist-fight, so we consulted Atticus. Our father said we were both right.

Being Southerners, it was a source of shame to some members of the family that we had no recorded ancestors on either side of the Battle of Hastings. All we had was Simon Finch, a fur-trapping apothecary from Cornwall whose piety was exceeded only by his stinginess. In England, Simon was irritated by the persecution of those who called themselves Methodists at the hands of their more liberal brethren, and as Simon called himself a Methodist, he worked his way across the Atlantic to Philadelphia, thence to Jamaica, thence to Mobile, and up the Saint Stephens. Mindful of John Wesleys strictures on the use of many words in buying and selling, Simon made a pile practicing medicine, but in this pursuit he was unhappy lest he be tempted into doing what he knew was not for the glory of God, as the putting on of gold and costly apparel. So Simon, having forgotten his teachers dictum on the possession of human chattels, bought three slaves and with their aid established a homestead on the banks of the Alabama River some forty miles above Saint Stephens. He returned to Saint Stephens only once, to find a wife, and with her established a line that ran high to daughters. Simon lived to an impressive age and died rich.
    """

    cleaned_text = ''.join([c for c in english_text if ord(c) < 128])
    data = cleaned_text.encode('ascii')

    print(f"английский текст из {len(cleaned_text)} символов")

    symbol_bytes_list = [1, 2, 3, 4]

    results = analyze_entropy_by_symbol_len(data, symbol_bytes_list)
    plot_entropy_vs_symbol_len(results)


if __name__ == "__main__":
    test_data = b"abracadabra"
    entropy = calculate_entropy(test_data, 1)
    print(f"энтропия 'abracadabra': {entropy:.4f}")
    print()

    test_entropy_for_english_text()
```

huffman.py:

```
"""
кодирование хаффмана
задание 2, пункт 1.4
"""


class HuffmanNode:
    def __init__(self, symbol=None, freq=0, left=None, right=None):
        self.symbol = symbol
        self.freq = freq
        self.left = left
        self.right = right

    def is_leaf(self):
        return self.left is None and self.right is None


def build_huffman_tree(freqs):
    """построение дерева хаффмана"""
    nodes = []
    for symbol, freq in freqs.items():
        nodes.append(HuffmanNode(symbol=symbol, freq=freq))

    def sort_nodes(nodes):
        for i in range(len(nodes)):
            for j in range(i + 1, len(nodes)):
                if nodes[i].freq > nodes[j].freq:
                    nodes[i], nodes[j] = nodes[j], nodes[i]
        return nodes

    while len(nodes) > 1:
        nodes = sort_nodes(nodes)
        left = nodes.pop(0)
        right = nodes.pop(0)
        parent = HuffmanNode(freq=left.freq + right.freq, left=left, right=right)
        nodes.append(parent)

    return nodes[0] if nodes else None


```

```python
def generate_codes(node, code="", codes=None):
    """генерация кодов хаффмана"""
    if codes is None:
        codes = {}

    if node.is_leaf():
        codes[node.symbol] = code
        return codes

    if node.left:
        generate_codes(node.left, code + "0", codes)
    if node.right:
        generate_codes(node.right, code + "1", codes)
    return codes


def huffman_encode(data):
    """кодирование хаффмана"""
    if not data:
        return b'', 0

    freqs = {}
    for byte in data:
        freqs[byte] = freqs.get(byte, 0) + 1

    tree = build_huffman_tree(freqs)
    codes = generate_codes(tree)

    bits = ""
    for byte in data:
        bits += codes[byte]

    padding = (8 - len(bits) % 8) % 8
    bits += "0" * padding

    encoded = bytearray()
    for i in range(0, len(bits), 8):
        byte_val = int(bits[i:i + 8], 2)
        encoded.append(byte_val)

    return bytes(encoded), padding

```

```
83
84  def huffman_decode(encoded_data, tree, padding):
85      """декодирование хаффмана"""
86      if not encoded_data:
87          return b''
88
89      bits = ""
90      for byte in encoded_data:
91          bits += format(byte, '08b')
92
93      if padding > 0:
94          bits = bits[:-padding]
95
96      result = bytearray()
97      node = tree
98
99      for bit in bits:
100         if bit == '0':
101             node = node.left
102         else:
103             node = node.right
104
105         if node.is_leaf():
106             result.append(node.symbol)
107             node = tree
108
109     return bytes(result)
110
111
112 def arithmetic_encode(data):
113     """арифметическое кодирование"""
114     if not data:
115         return 0.0, 0.0, 0, {}
116
117     freqs = {}
118     for b in data:
119         freqs[b] = freqs.get(b, 0) + 1
120
121     total = len(data)
122
123     cum_probs = {}
124     cum = 0.0
```

```
    for sym in sorted(freqs.keys()):
        prob = freqs[sym] / total
        cum_probs[sym] = (cum, cum + prob)
        cum += prob

    low = 0.0
    high = 1.0

    for b in data:
        range_width = high - low
        low_b, high_b = cum_probs[b]
        low = low + range_width * low_b
        high = low + range_width * (high_b - low_b)

    code = (low + high) / 2

    return code, low, high, len(data), cum_probs


def arithmetic_decode(code, low, high, data_len, cum_probs):
    """арифметическое декодирование"""
    result = bytearray()
    current_low = 0.0
    current_high = 1.0
    symbols = sorted(cum_probs.keys())

    for _ in range(data_len):
        range_width = current_high - current_low
        value = (low + high) / 2

        for sym in symbols:
            low_b, high_b = cum_probs[sym]
            sym_low = current_low + range_width * low_b
            sym_high = current_low + range_width * high_b

            if sym_low <= value < sym_high:
                result.append(sym)
                current_low = sym_low
                current_high = sym_high
                break

    return bytes(result)
```

```
167
168
169 if __name__ == "__main__":
170     test_data = bytes([0x42, 0x41, 0x4E, 0x41, 0x4E, 0x41])
171     encoded, padding = huffman_encode(test_data)
172     freqs = {}
173     for b in test_data:
174         freqs[b] = freqs.get(b, 0) + 1
175     tree = build_huffman_tree(freqs)
176     decoded = huffman_decode(encoded, tree, padding)
177     print(f"BANANA: вход={test_data}\nдекодировано={decoded}")
178     print()
179
180     print("арифметическое кодирование")
181     test_arith = bytes([0x42, 0x41, 0x4E, 0x41, 0x4E, 0x41])
182     code, low, high, data_len, cum_probs = arithmetic_encode(test_arith)
183     print(f"code = {code:.20f}")
184     print(f"low = {low:.20f}")
185     print(f"high = {high:.20f}")
186
187     decoded_arith = arithmetic_decode(code, low, high, data_len, cum_probs)
188     print(f"декодировано: {decoded_arith}")
```

mtf.py:

```
1  """
2  Move-To-Front (MTF) преобразование
3  задание 2, пункт 1.3
4  """
5
6
7  def mtf_encode(data):
8      """MTF кодирование"""
9      if not data:
10         return []
11
12     symbols = list(range(256))
13     result = []
14
15     for byte in data:
16         pos = symbols.index(byte)
17         result.append(pos)
```

```
18
19          symbols.pop(pos)
20          symbols.insert(0, byte)
21
22      return result
23
24
25  def mtf_decode(indices):
26      """MTF декодирование"""
27      if not indices:
28          return b''
29
30      symbols = list(range(256))
31      result = bytearray()
32
33      for idx in indices:
34          byte = symbols[idx]
35          result.append(byte)
36
37          symbols.pop(idx)
38          symbols.insert(0, byte)
39
40      return bytes(result)
41
42
43  def mtf_encode_utf8(text):
44      """MTF кодирование для UTF-8 текста"""
45      codepoints = [ord(ch) for ch in text]
46      symbols = []
47      result = []
48
49      for code in codepoints:
50          if code in symbols:
51              pos = symbols.index(code)
52          else:
53              symbols.append(code)
54              pos = len(symbols) - 1
55
56          result.append(pos)
57          symbols.pop(pos)
58          symbols.insert(0, code)
59
```

```
    return result


if __name__ == "__main__":
    test_data = bytes([0x42, 0x41, 0x4E, 0x41, 0x4E, 0x41])  # BANANA
    encoded = mtf_encode(test_data)
    decoded = mtf_decode(encoded)
    print(f"вход: {test_data}")
    print(f"выход MTF: {encoded}")
    print(f"декодировано: {decoded}")

    print()
    test_data2 = bytes([0x41, 0x41, 0x41, 0x41, 0x41, 0x42, 0x42, 0x42, 0x42, 0x42])
    encoded2 = mtf_encode(test_data2)
    decoded2 = mtf_decode(encoded2)
    print(f"вход: {test_data2}")
    print(f"выход MTF: {encoded2}")
    print(f"декодировано: {decoded2}")
```

Задание 3:

lz77.py:

```
"""
LZ77 и LZSS алгоритмы
задание 3, пункт 2.1-2.2
"""


def lz77_encode(data, buffer_size=4096):
    """LZ77 кодирование"""
    if not data:
        return b''

    result = bytearray()
    i = 0
    n = len(data)

    while i < n:
        best_offset = 0
        best_len = 0

        start = max(0, i - buffer_size)
```

```
21
22          for offset in range(1, i - start + 1):
23              length = 0
24              while (i + length < n and
25                     length < 255 and
26                     data[i + length] == data[i - offset + length]):
27                  length += 1
28
29              if length > best_len:
30                  best_len = length
31                  best_offset = offset
32
33          if i + best_len < n:
34              next_char = data[i + best_len]
35          else:
36              next_char = 0
37
38          result.append((best_offset >> 8) & 0xFF)
39          result.append(best_offset & 0xFF)
40          result.append(best_len)
41          result.append(next_char)
42
43          i += best_len + 1
44
45      return bytes(result)
46
47
48  def lz77_decode(data):
49      """LZ77 декодирование"""
50      if not data:
51          return b''
52
53      result = bytearray()
54      i = 0
55      n = len(data)
56
57      while i + 3 < n:
58          offset = (data[i] << 8) | data[i + 1]
59          length = data[i + 2]
60          next_char = data[i + 3]
61          i += 4
62
```

```
        if length > 0:
            start = len(result) - offset
            for j in range(length):
                result.append(result[start + j])

        if next_char != 0:
            result.append(next_char)

    return bytes(result)



if __name__ == "__main__":
    test_data = b"abracadabra"
    print(f"\nисходные данные: {test_data.decode()}")
    print(f"размер: {len(test_data)} байт")

    encoded = lz77_encode(test_data, 256)
    print(f"\nLZ77 закодировано: {len(encoded)} байт")
    print(f"коэффициент сжатия: {len(test_data) / len(encoded):.2f}")

    decoded = lz77_decode(encoded)
    print(f"декодировано: {decoded.decode()}")

```

lz78_lzw.py:

```
"""
LZ78 и LZW алгоритмы
задание 3, пункт 2.3-2.4
"""


def lz78_encode(data, max_dict_size=4096):
    """LZ78 кодирование"""
    if not data:
        return b'', 0

    dictionary = {}
    dict_size = 1
    result = bytearray()
    i = 0
```

```
    n = len(data)

    while i < n:
        current = b''
        idx = 0
        j = i

        while j < n:
            candidate = current + bytes([data[j]])
            if candidate in dictionary:
                current = candidate
                idx = dictionary[current]
                j += 1
            else:
                break

        if current:
            if j >= n:
                result.append((idx >> 8) & 0xFF)
                result.append(idx & 0xFF)
                result.append(0)
                i = j
            else:
                next_char = data[j]
                result.append((idx >> 8) & 0xFF)
                result.append(idx & 0xFF)
                result.append(next_char)

                new_phrase = current + bytes([next_char])
                if dict_size < max_dict_size and new_phrase not in dictionary:
                    dictionary[new_phrase] = dict_size
                    dict_size += 1

                i = j + 1
        else:
            result.append(0)
            result.append(0)
            result.append(data[i])

            if dict_size < max_dict_size and bytes([data[i]]) not in dictionary:
                dictionary[bytes([data[i]])] = dict_size
                dict_size += 1
```

```
            i += 1

    return bytes(result), len(data)


def lz78_decode(data, orig_len, max_dict_size=4096):
    """LZ78 декодирование"""
    if not data:
        return b''

    dictionary = {}
    dict_size = 1
    result = bytearray()
    i = 0
    n = len(data)

    while i + 2 < n and len(result) < orig_len:
        idx = (data[i] << 8) | data[i + 1]
        next_char = data[i + 2]
        i += 3

        if idx == 0:
            if next_char == 0:
                continue
            phrase = bytes([next_char])
        else:
            if idx in dictionary:
                phrase = dictionary[idx] + bytes([next_char]) if next_char != 0 else
    dictionary[idx]
            else:
                phrase = bytes([next_char]) if next_char != 0 else b''

        if len(phrase) == 0:
            continue

        if len(result) + len(phrase) > orig_len:
            phrase = phrase[:orig_len - len(result)]

        result.extend(phrase)

        if dict_size < max_dict_size and idx != 0:
```

```
            dictionary[dict_size] = phrase
            dict_size += 1
        elif dict_size < max_dict_size and idx == 0 and next_char != 0:
            if bytes([next_char]) not in dictionary.values():
                dictionary[dict_size] = bytes([next_char])
                dict_size += 1

    return bytes(result)


def lzw_encode(data, max_dict_size=4096):
    """LZW кодирование"""
    if not data:
        return b'', 0

    dictionary = {}
    for i in range(256):
        dictionary[bytes([i])] = i

    dict_size = 256
    result = bytearray()
    current = b''

    for byte in data:
        current_byte = bytes([byte])
        new_phrase = current + current_byte

        if new_phrase in dictionary:
            current = new_phrase
        else:
            code = dictionary[current]
            result.append((code >> 8) & 0xFF)
            result.append(code & 0xFF)

            if dict_size < max_dict_size:
                dictionary[new_phrase] = dict_size
                dict_size += 1

            current = current_byte

    if current:
        code = dictionary[current]
```

```
        result.append((code >> 8) & 0xFF)
        result.append(code & 0xFF)

    return bytes(result), len(data)


def lzw_decode(data, max_dict_size=4096):
    """LZW декодирование"""
    if not data:
        return b''

    dictionary = {}
    for i in range(256):
        dictionary[i] = bytes([i])

    dict_size = 256
    result = bytearray()

    i = 0
    n = len(data)

    if i + 1 >= n:
        return b''
    code = (data[i] << 8) | data[i + 1]
    i += 2

    if code not in dictionary:
        return b''

    current = dictionary[code]
    result.extend(current)

    while i + 1 < n:
        code = (data[i] << 8) | data[i + 1]
        i += 2

        if code in dictionary:
            entry = dictionary[code]
        elif code == dict_size:
            entry = current + bytes([current[0]])
        else:
            break
```

```
        result.extend(entry)

        if dict_size < max_dict_size:
            dictionary[dict_size] = current + bytes([entry[0]])
            dict_size += 1

        current = entry

    return bytes(result)


if __name__ == "__main__":
    print("LZ78")
    test_data = b"abracadabra"
    encoded, orig_len = lz78_encode(test_data)
    decoded = lz78_decode(encoded, orig_len)
    print(f"исходные: {test_data.decode()}")
    print(f"закодировано: {len(encoded)} байт")
    print(f"декодировано: {decoded.decode()}")

```

lzss.py:

```
def lzss_encode(data, window_size=4096):
    """LZSS кодирование"""
    if not data:
        return b'', 0

    output_bits = []
    output_bytes = []
    i = 0
    n = len(data)

    while i < n:
        search_start = max(0, i - window_size)
        best_len = 0
        best_pos = 0

        for pos in range(search_start, i):
            length = 0
            while (i + length < n and
```

```
                    length < 255 and
                    data[pos + length] == data[i + length]):
                length += 1

            if length > best_len:
                best_len = length
                best_pos = i - pos

        if best_len >= 3:
            output_bits.append(1)
            output_bytes.append((best_pos >> 8) & 0xFF)
            output_bytes.append(best_pos & 0xFF)
            output_bytes.append(best_len)
            i += best_len
        else:
            output_bits.append(0)
            output_bytes.append(data[i])
            i += 1

    packed_flags = bytearray()
    for j in range(0, len(output_bits), 8):
        byte = 0
        for k in range(8):
            if j + k < len(output_bits) and output_bits[j + k]:
                byte |= (1 << (7 - k))
        packed_flags.append(byte)

    result = bytearray()
    result.append((len(packed_flags) >> 8) & 0xFF)
    result.append(len(packed_flags) & 0xFF)
    result.extend(packed_flags)
    result.extend(output_bytes)

    return bytes(result), len(data)


def lzss_decode(data, orig_len, window_size=4096):
    """LZSS декодирование"""
    if not data:
        return b''

    flags_len = (data[0] << 8) | data[1]
```

```
    flags_data = data[2:2 + flags_len]
    bytes_data = data[2 + flags_len:]

    flags = []
    for byte in flags_data:
        for i in range(8):
            flags.append((byte >> (7 - i)) & 1)

    result = bytearray()
    byte_pos = 0

    for flag in flags:
        if len(result) >= orig_len:
            break

        if flag == 0:
            if byte_pos >= len(bytes_data):
                break
            result.append(bytes_data[byte_pos])
            byte_pos += 1
        else:
            if byte_pos + 2 >= len(bytes_data):
                break
            offset = (bytes_data[byte_pos] << 8) | bytes_data[byte_pos + 1]
            length = bytes_data[byte_pos + 2]
            byte_pos += 3

            start = len(result) - offset
            if start < 0:
                start = 0

            for j in range(length):
                if len(result) >= orig_len:
                    break
                result.append(result[start + j])

    return bytes(result)


if __name__ == "__main__":
    test1 = b"abracadabra"
    print(f"Test1: {test1.decode()}")
```

```
103
104     enc1, orig1 = lzss_encode(test1)
105     dec1 = lzss_decode(enc1, orig1)
106
107     print(f"размер: исходный={len(test1)}, сжатый={len(enc1)}, коэфф
    ={len(test1)/len(enc1):.2f}")
108     print(f"декодировано верно: {test1 == dec1}")
109
110     test2 = b"aaaaabbbbbccccccaaaaabbbbb"
111     print(f"\nTest2: {test2.decode()}")
112
113     enc2, orig2 = lzss_encode(test2)
114     dec2 = lzss_decode(enc2, orig2)
115
116     print(f"размер: исходный={len(test2)}, сжатый={len(enc2)}, коэфф
    ={len(test2)/len(enc2):.2f}")
117     print(f"декодировано верно: {test2 == dec2}")
118
119     print()
120
121     test3 = bytes([0]) * 1000
122     enc3, orig3 = lzss_encode(test3)
123     dec3 = lzss_decode(enc3, orig3)
124
125     print(f"1000 нулей: сжатый размер={len(enc3)}, коэфф={len(test3)/len(enc3):.2f}")
126     print(f"декодировано верно: {test3 == dec3}")
127
```

suffix_array.py:

```
1  """
2  суффиксный массив для BWT
3  задание 3, пункт 1
4  """
5
6
7  def build_suffix_array(data):
8      """простое построение суффиксного массива"""
9      n = len(data)
10     suffixes = []
11
12     for i in range(n):
```

```
        suffixes.append((data[i:], i))

    for i in range(n):
        for j in range(i + 1, n):
            if suffixes[i][0] > suffixes[j][0]:
                suffixes[i], suffixes[j] = suffixes[j], suffixes[i]

    suffix_array = [suffixes[i][1] for i in range(n)]
    return suffix_array


def suffix_array_to_bwt(suffix_array, original_data):
    """преобразование суффиксного массива в BWT"""
    if not suffix_array or not original_data:
        return b''

    n = len(original_data)
    result = bytearray()

    for i in range(n):
        pos = suffix_array[i]
        if pos == 0:
            result.append(original_data[n - 1])
        else:
            result.append(original_data[pos - 1])

    return bytes(result)


def build_suffix_array_manber_myers(data):
    """построение суффиксного массива алгоритмом манбера-майерса"""
    n = len(data)
    if n == 0:
        return []

    rank = [data[i] for i in range(n)]
    tmp = [0] * n
    sa = list(range(n))

    k = 1
    while True:
        sa.sort(key=lambda x: (rank[x], rank[x + k] if x + k < n else -1))
```

```
55
56          tmp[sa[0]] = 0
57          for i in range(1, n):
58              prev = sa[i - 1]
59              curr = sa[i]
60              prev_val = rank[prev + k] if prev + k < n else -1
61              curr_val = rank[curr + k] if curr + k < n else -1
62
63              if rank[prev] == rank[curr] and prev_val == curr_val:
64                  tmp[curr] = tmp[prev]
65              else:
66                  tmp[curr] = tmp[prev] + 1
67
68          rank, tmp = tmp, rank
69
70          if rank[sa[-1]] == n - 1:
71              break
72          k <<= 1
73
74      return sa
75
76
77  def bwt_from_suffix_array(data, suffix_array):
78      """BWT из суффиксного массива"""
79      n = len(data)
80      result = bytearray()
81
82      for pos in suffix_array:
83          if pos == 0:
84              result.append(data[-1])
85          else:
86              result.append(data[pos - 1])
87
88      return bytes(result)
89
90
91  def bwt_encode_effective(data):
92      """эффективное BWT с использованием суффиксного массива"""
93      if not data:
94          return b'', 0
95
96      suffix_array = build_suffix_array_manber_myers(data)
```

```
    bwt = bwt_from_suffix_array(data, suffix_array)

    for i, pos in enumerate(suffix_array):
        if pos == 0:
            index = i
            break

    return bwt, index


if __name__ == "__main__":
    data = bytes([0x62, 0x61, 0x6e, 0x61, 0x6e, 0x61])  # "banana"
    print(f"\nисходные данные: {' '.join(f'0x{b:02x}' for b in data)}")
    print(data.decode('ascii'))

    suffix_array = build_suffix_array(data)
    print(f"\nсуффиксный массив: {suffix_array}")

    bwt_result = suffix_array_to_bwt(suffix_array, data)
    print(f"BWT через суффиксный массив: {' '.join(f'0x{b:02x}' for b in bwt_result)}")

```

Задание 4:

cannonical_huffman.py:

```
"""
канонические коды хаффмана
задание 4, пункт 2
"""


def build_huffman_freqs(data):
    """построение частот символов"""
    freqs = {}
    for byte in data:
        freqs[byte] = freqs.get(byte, 0) + 1
    return freqs


def build_huffman_tree(freqs):
    """построение дерева хаффмана"""
    nodes = []
```

```
    for symbol, freq in freqs.items():
        nodes.append([freq, symbol, None, None])

    while len(nodes) > 1:
        nodes.sort(key=lambda x: x[0])
        left = nodes.pop(0)
        right = nodes.pop(0)
        parent = [left[0] + right[0], None, left, right]
        nodes.append(parent)

    return nodes[0] if nodes else None


def get_code_lengths(node, depth=0, lengths=None):
    """получение длин кодов хаффмана"""
    if lengths is None:
        lengths = {}

    if node[1] is not None:
        lengths[node[1]] = depth
    else:
        if node[2]:
            get_code_lengths(node[2], depth + 1, lengths)
        if node[3]:
            get_code_lengths(node[3], depth + 1, lengths)

    return lengths


def build_can_codes(code_lengths):
    """построение канонических кодов по длинам"""
    symbols = sorted(code_lengths.items(), key=lambda x: (x[1], x[0]))

    codes = {}
    current_code = 0
    current_len = symbols[0][1]

    for symbol, length in symbols:
        if length > current_len:
            current_code <<= (length - current_len)
            current_len = length

```

```
        code_bits = bin(current_code)[2:].zfill(length)
        codes[symbol] = code_bits
        current_code += 1

    return codes


def can_huffman_encode(data):
    """кодирование каноническими кодами хаффмана"""
    if not data:
        return b'', {}, 0

    freqs = build_huffman_freqs(data)
    tree = build_huffman_tree(freqs)
    code_lengths = get_code_lengths(tree)
    codes = build_can_codes(code_lengths)

    bits = ""
    for byte in data:
        bits += codes[byte]

    padding = (8 - len(bits) % 8) % 8
    bits += "0" * padding

    encoded = bytearray()
    for i in range(0, len(bits), 8):
        encoded.append(int(bits[i:i + 8], 2))

    return bytes(encoded), code_lengths, padding


def can_huffman_decode(encoded_data, code_lengths, padding, data_len):
    """декодирование канонических кодов хаффмана"""
    if not encoded_data:
        return b''

    codes = build_can_codes(code_lengths)
    reverse_codes = {v: k for k, v in codes.items()}

    bits = ""
    for byte in encoded_data:
        bits += format(byte, '08b')
```

```
    if padding > 0:
        bits = bits[:-padding]

    result = bytearray()
    current_code = ""

    for bit in bits:
        current_code += bit
        if current_code in reverse_codes:
            result.append(reverse_codes[current_code])
            current_code = ""

    return bytes(result)


def save_canonical_huffman(data, output_path):
    """сохранение канонически закодированных данных"""
    if not data:
        return 0, 0

    encoded, code_lengths, padding = can_huffman_encode(data)

    with open(output_path, 'wb') as f:
        f.write(len(code_lengths).to_bytes(4, 'little'))
        for symbol, length in sorted(code_lengths.items()):
            f.write(symbol.to_bytes(1, 'little'))
            f.write(length.to_bytes(1, 'little'))
        f.write(padding.to_bytes(1, 'little'))
        f.write(len(data).to_bytes(4, 'little'))
        f.write(encoded)

    return len(data), len(encoded) + 4 + 2 * len(code_lengths) + 1 + 4


def load_canonical_huffman(input_path):
    """загрузка канонически закодированных данных"""
    with open(input_path, 'rb') as f:
        num_symbols = int.from_bytes(f.read(4), 'little')
        code_lengths = {}
        for _ in range(num_symbols):
            symbol = f.read(1)[0]
```

```
            length = f.read(1)[0]
            code_lengths[symbol] = length
        padding = f.read(1)[0]
        data_len = int.from_bytes(f.read(4), 'little')
        encoded = f.read()

    return can_huffman_decode(encoded, code_lengths, padding, data_len)


if __name__ == "__main__":
    test1 = b"abracadabra"
    print(test1.decode())
    print(f"размер: {len(test1)} байт")
    encoded1, lengths1, padding1 = can_huffman_encode(test1)
    print(f"сжатый размер: {len(encoded1)} байт")
    print(f"коэффициент: {len(test1) / len(encoded1):.2f}")
    print(f"метаданные (длины кодов): {dict(sorted(lengths1.items()))}")
    decoded1 = can_huffman_decode(encoded1, lengths1, padding1, len(test1))
    print(f"верно: {test1 == decoded1}")
```

efficient_bwt.py:

```
"""
эффективное преобразование барроузауиллера (BWT)
на основе суффиксного массива.
"""

def build_suffix_array(data):
    n = len(data)

    # начальная сортировка по одному символу
    sa = list(range(n))
    rank = list(data)
    tmp = [0] * n

    k = 1

    while k < n:

        # сортировка по паре:
        # (rank[i], rank[i + k])
        sa.sort(
```

```
            key=lambda i: (
                rank[i],
                rank[i + k] if i + k < n else -1
            )
        )

        # пересчет классов эквивалентности
        tmp[sa[0]] = 0

        for i in range(1, n):

            prev = sa[i - 1]
            curr = sa[i]

            prev_rank = (
                rank[prev],
                rank[prev + k] if prev + k < n else -1
            )

            curr_rank = (
                rank[curr],
                rank[curr + k] if curr + k < n else -1
            )

            tmp[curr] = tmp[prev] + (prev_rank != curr_rank)

        rank[:] = tmp[:]

        # все ранги уникальны
        if rank[sa[-1]] == n - 1:
            break

        k *= 2

    return sa


def bwt_encode_suffix_array(data):
    if not data:
        return b'', 0

    n = len(data)
```

```
    # строим суффиксный массив
    sa = build_suffix_array(data)

    last_column = bytearray()
    primary_idx = 0

    for i, suffix_pos in enumerate(sa):

        # символ перед суффиксом
        last_column.append(data[suffix_pos - 1])

        # исходная строка соответствует суффиксу с позиции 0
        if suffix_pos == 0:
            primary_idx = i

    return bytes(last_column), primary_idx


if __name__ == "__main__":

    test_data = b"banana"

    print("исходные данные:", test_data)

    bwt_data, idx = bwt_encode_suffix_array(test_data)

    print("BWT:", bwt_data)
    print("индекс:", idx)
```

Обработка:

task1:

```
"""
задание 1: зависимость энтропии от размера блока (BWT + MTF)
"""

import time
import matplotlib.pyplot as plt
import sys
sys.path.append('..')

```

```python
from exercise2.bwt import bwt_encode_block
from exercise2.mtf import mtf_encode
from exercise2.entropy import calculate_entropy


def bwt_mtf_block(data, block_size):
    """применение BWT + MTF с блочной обработкой"""
    result = bytearray()
    n = len(data)
    i = 0

    while i < n:
        block = data[i:i + block_size]
        bwt_block, _ = bwt_encode_block(block)
        mtf_indices = mtf_encode(bwt_block)
        result.extend(mtf_indices)
        i += block_size
    return bytes(result)


def analyze_bwt_mtf_entropy(data, block_sizes):
    """анализ энтропии после BWT + MTF"""
    results = {'block_sizes': [], 'entropy': []}

    print(f"\n{'размер блока':<15} {'энтропия':<12}")
    print("-" * 30)

    for block_size in block_sizes:
        transformed_data = bwt_mtf_block(data, block_size)
        entropy_after = calculate_entropy(transformed_data, 1)

        results['block_sizes'].append(block_size)
        results['entropy'].append(entropy_after)

        print(f"{block_size:<15} {entropy_after:<12.4f}")

    return results


def plot_entropy_vs_block_size(results, original_entropy):
    """график зависимости энтропии от размера блока"""
    plt.figure(figsize=(10, 6))
```

```
52
53      plt.plot(results['block_sizes'], results['entropy'], 'b-o', linewidth=2, markersize=6)
54      plt.axhline(y=original_entropy, color='r', linestyle='--', label=f'исходная энтропия:
        {original_entropy:.4f}')
55
56      plt.xlabel('размер блока (байт)', fontsize=12)
57      plt.ylabel('энтропия (бит/символ)', fontsize=12)
58      plt.title('зависимость энтропии от размера блока (BWT + MTF)', fontsize=14)
59      plt.legend()
60      plt.grid(True, alpha=0.3)
61
62      plt.tight_layout()
63      plt.savefig('entropy_vs_block_size.png', dpi=150)
64      plt.show()
65
66
67  def test_on_file(filepath, max_bytes=None):
68      """тестирование на файле"""
69      print(f"файл: {filepath}")
70
71      with open(filepath, 'rb') as f:
72          if max_bytes:
73              data = f.read(max_bytes)
74          else:
75              data = f.read()
76
77      print(f"загружено {len(data)} байт")
78
79      block_sizes = [64, 128, 256, 512, 1024, 2048, 4096, 8192]
80      print("\nзадание 1: зависимость энтропии от размера блока (BWT + MTF)")
81
82      original_entropy = calculate_entropy(data, 1)
83      print(f"исходная энтропия: {original_entropy:.4f}")
84
85      results = analyze_bwt_mtf_entropy(data, block_sizes)
86      plot_entropy_vs_block_size(results, original_entropy)
87
88      return results
89
90
91  if __name__ == "__main__":
92      file_path = "enwik7"
```

```
    results = test_on_file(file_path, max_bytes=100 * 1024)
```

task2:

```
"""
задание 2: зависимость коэффициента сжатия от размера буфера (LZSS)
"""

import time
import matplotlib.pyplot as plt
import sys
sys.path.append('..')

from exercise3.lz77 import lzss_encode


def analyze_lzss_buffer_size(data, buffer_sizes):
    """анализ зависимости коэффициента сжатия от размера буфера"""
    results = {'buffer_sizes': [], 'compression_ratio': [], 'time': []}

    print(f"\n{'размер буфера':<15} {'коэффициент':<15} {'время (с)':<10}")
    print("-" * 45)

    for buffer_size in buffer_sizes:
        start_time = time.time()
        encoded, _ = lzss_encode(data, window_size=buffer_size)
        elapsed_time = time.time() - start_time

        compressed_size = len(encoded)
        ratio = len(data) / compressed_size if compressed_size > 0 else 0

        results['buffer_sizes'].append(buffer_size)
        results['compression_ratio'].append(ratio)
        results['time'].append(elapsed_time)

        print(f"{buffer_size:<15} {ratio:<15.3f} {elapsed_time:<10.4f}")

    return results


def plot_lzss_buffer(results):
    """график зависимости коэффициента сжатия от размера буфера"""
```

```
    plt.figure(figsize=(10, 6))

    plt.plot(results['buffer_sizes'], results['compression_ratio'], 'b-o', linewidth=2,
    markersize=6)

    plt.xlabel('размер буфера (байт)', fontsize=12)
    plt.ylabel('коэффициент сжатия', fontsize=12)
    plt.title('зависимость коэффициента сжатия LZSS от размера буфера', fontsize=14)
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('lzss_buffer_vs_ratio.png', dpi=150)
    plt.show()


def test_lzss_on_file(filepath, max_bytes=None):
    """тестирование на файле"""
    print(f"файл: {filepath}")

    with open(filepath, 'rb') as f:
        if max_bytes:
            data = f.read(max_bytes)
        else:
            data = f.read()

    print(f"загружено {len(data)} байт")

    buffer_sizes = [256, 512, 1024, 2048, 4096, 8192, 16384, 32768]
    print("\nзадание 2: зависимость коэффициента сжатия от размера буфера (LZSS)")

    results = analyze_lzss_buffer_size(data, buffer_sizes)
    plot_lzss_buffer(results)

    return results


if __name__ == "__main__":
    file_path = "enwik7"
    results = test_lzss_on_file(file_path, max_bytes=100 * 1024)
```

task3:

```python
"""
задание 3: зависимость коэффициента сжатия от размера словаря (LZW)
"""

import matplotlib.pyplot as plt
import sys
sys.path.append('..')

from exercise3.lz78_lzw import lzw_encode


def analyze_lzw_dict_size(data, dict_sizes):
    """анализ зависимости коэффициента сжатия от размера словаря"""
    results = {'dict_sizes': [], 'compression_ratio': []}

    print(f"\n{'размер словаря':<15} {'коэффициент':<20}")
    print("-" * 40)

    for dict_size in dict_sizes:
        encoded, orig_len = lzw_encode(data, max_dict_size=dict_size)

        compressed_size = len(encoded)
        ratio = len(data) / compressed_size if compressed_size > 0 else 0

        results['dict_sizes'].append(dict_size)
        results['compression_ratio'].append(ratio)

        print(f"{dict_size:<15} {ratio:<20.3f}")

    return results


def plot_ratio_vs_dict_size(results):
    """график зависимости коэффициента сжатия от размера словаря"""
    plt.figure(figsize=(10, 6))

    plt.plot(results['dict_sizes'], results['compression_ratio'], 'orange', marker='o',
    linewidth=2, markersize=6)

    plt.xlabel('размер словаря', fontsize=12)
    plt.ylabel('коэффициент сжатия', fontsize=12)
    plt.title('зависимость коэффициента сжатия LZW от размера словаря', fontsize=14)
```

```
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('lzw_dict_vs_ratio.png', dpi=150)
    plt.show()


def test_lzw_on_file(filepath, max_bytes=None):
    """тестирование на файле"""
    print(f"файл: {filepath}")

    with open(filepath, 'rb') as f:
        if max_bytes:
            data = f.read(max_bytes)
        else:
            data = f.read()

    print(f"загружено {len(data)} байт")

    dict_sizes = [256, 512, 1024, 2048, 4096, 8192, 16384]
    print("\nзадание 3: зависимость коэффициента сжатия от размера словаря (LZW)")

    results = analyze_lzw_dict_size(data, dict_sizes)
    plot_ratio_vs_dict_size(results)

    return results


if __name__ == "__main__":
    file_path = "enwik7"
    results = test_lzw_on_file(file_path, max_bytes=100 * 1024)
```

task4:

```
import time
import os
from exercise1.rle import rle_encode, rle_decode, rle_encode_with_len
from exercise2.bwt import bwt_encode_block, bwt_decode_fast
from exercise2.mtf import mtf_encode, mtf_decode
from exercise2.huffman import huffman_encode, huffman_decode, build_huffman_tree
from exercise3.lzss import lzss_encode, lzss_decode
from exercise3.lz78_lzw import lzw_encode, lzw_decode
```

```python

from exercise2.entropy import calculate_entropy


def compressor_huffman(data):
    encoded, tree = huffman_encode(data)  # распаковываем кортеж
    return encoded

def compressor_rle(data):
    return rle_encode(data)
    # return rle_encode_with_len(data)
    #


def compressor_bwt_rle(data, block_size=4096):
    """BWT + RLE с правильной обработкой блоков"""
    if not data:
        return b''

    result = bytearray()
    n = len(data)

    # заголовок: исходный размер и размер блока
    result.extend(n.to_bytes(4, 'little'))
    result.extend(block_size.to_bytes(4, 'little'))

    i = 0
    while i < n:
        # берем блок (последний может быть меньше)
        block = data[i:i + block_size]
        block_len = len(block)

        # BWT
        bwt_block, idx = bwt_encode_block(block)

        # RLE
        rle_block = rle_encode(bwt_block)

        # сохраняем: [BWT индекс (4)] [размер блока (4)] [размер RLE (4)] [RLE данные]
        result.extend(idx.to_bytes(4, 'little'))
        result.extend(block_len.to_bytes(4, 'little'))
        result.extend(len(rle_block).to_bytes(4, 'little'))
```

```
        result.extend(rle_block)

        i += block_size

    return bytes(result)


def compressor_bwt_mtf_huffman(data, block_size=4096):
    result = bytearray()
    i = 0
    n = len(data)

    while i < n:
        block = data[i:i + block_size]
        if len(block) < block_size:
            block = block + b'\x00' * (block_size - len(block))
        bwt_block, idx = bwt_encode_block(block)
        # сохраняем индекс (4 байта)
        result.extend(idx.to_bytes(4, 'little'))
        mtf_indices = mtf_encode(bwt_block)
        mtf_bytes = bytes(mtf_indices)

        # исправлено: распаковываем результат
        huff_encoded, tree = huffman_encode(mtf_bytes)
        result.extend(huff_encoded)
        i += block_size

    return bytes(result)


def compressor_bwt_mtf(data, block_size=4096):
    result = bytearray()
    i = 0
    n = len(data)

    while i < n:
        block = data[i:i + block_size]
        if len(block) < block_size:
            block = block + b'\x00' * (block_size - len(block))
        bwt_block, idx = bwt_encode_block(block)
        # сохр индекс (4 байта)
        result.extend(idx.to_bytes(4, 'little'))
```

```
        mtf_indices = mtf_encode(bwt_block)
        mtf_bytes = bytes(mtf_indices)
        huff_encoded = huffman_encode(mtf_bytes)
        result.extend(huff_encoded)
        i += block_size

    return bytes(result)


def compressor_bwt_mtf_rle_huffman(data, block_size=4096):
    result = bytearray()
    i = 0
    n = len(data)

    while i < n:
        block = data[i:i + block_size]
        if len(block) < block_size:
            block = block + b'\x00' * (block_size - len(block))
        bwt_block, idx = bwt_encode_block(block)
        result.extend(idx.to_bytes(4, 'little'))
        mtf_indices = mtf_encode(bwt_block)
        mtf_bytes = bytes(mtf_indices)
        rle_block = rle_encode(mtf_bytes)

        # исправлено: распаковываем результат
        huff_encoded, tree = huffman_encode(rle_block)
        result.extend(huff_encoded)
        i += block_size

    return bytes(result)

def compressor_lzss(data, window_size=4096):
    encoded = lzss_encode(data, window_size)
    return encoded



def compressor_lzss_huffman(data, window_size=4096):
    lzss_compressed = lzss_encode(data, window_size)
    # исправлено: распаковываем результат
    huff_compressed, tree = huffman_encode(lzss_compressed)
    return huff_compressed
```

```
135
136
137 def compressor_lzw(data, dict_size=4096):
138     encoded, _ = lzw_encode(data, dict_size)
139     return encoded
140
141
142 def compressor_lzw_huffman(data, dict_size=4096):
143     lzw_compressed, _ = lzw_encode(data, dict_size)
144     # исправлено: распаковываем результат
145     huff_compressed, tree = huffman_encode(lzw_compressed)
146     return huff_compressed
147
148
149 def get_file_size_mb(size_bytes):
150     if size_bytes < 1024:
151         return f"{size_bytes} B"
152     elif size_bytes < 1024 * 1024:
153         return f"{size_bytes / 1024:.2f} KB"
154     else:
155         return f"{size_bytes / (1024 * 1024):.2f} MB"
156
157
158 def test_all_compressors(data, filename, description):
159     print(f"файл: {filename}")
160     print(f"описание: {description}")
161     print(f"исходный размер: {get_file_size_mb(len(data))} ({len(data)} байт)")
162     print(f"{'=' * 90}")
163
164     compressors = [
165         ("HA", compressor_huffman),
166         ("RLE", compressor_rle),
167         ("BWT + RLE", compressor_bwt_rle),
168
169         ("BWT + MTF + HA", compressor_bwt_mtf_huffman),
170         # ("BWT + MTF + HA", compressor_bwt_mtf),
171
172         # ("BWT + MTF + HA", compressor_bwt_mtf_huffman),
173
174
175
176         ("BWT + MTF + RLE + HA", compressor_bwt_mtf_rle_huffman),
```

```
        ("LZSS", compressor_lzss),
        ("LZSS + HA", compressor_lzss_huffman),
        ("LZW", compressor_lzw),
        ("LZW + HA", compressor_lzw_huffman),
    ]

    results = []
    entropy = calculate_entropy(data, 1)
    size = len(data)

    # if size < 1024:

    print(f"{filename:<25} {size:<15} {entropy:<12.4f}")
    print()

    print(f"\n{'компрессор':<25} {'сжатый размер':<18} {'коэффициент':<12} {'время
    (с)':<10}")
    print("-" * 75)

    for name, comp_func in compressors:
        start_time = time.time()
        compressed = comp_func(data)
        elapsed_time = time.time() - start_time

        compressed_size = len(compressed)
        ratio = compressed_size / len(data) if compressed_size > 0 else 0

        results.append({
            'name': name,
            'compressed_size': compressed_size,
            'ratio': ratio,
            'time': elapsed_time
        })

        size_str = get_file_size_mb(compressed_size)
        print(f"{name:<25} {size_str:<18} {ratio:<12.3f} {elapsed_time:<10.3f}")

    return results


def main():
    test_files = [
```

```
218
219         #("bw.raw", "черно-белое изображение, 1200x800"),
220         #("grey.raw", "изображение в оттенках серого, 1200x800"),
221         ("color.raw", "цветное изображение, 800x600"),
222         #(r"C:\Users\roman\PycharmProjects\algo_lab1\analysis\Haksli_200kb.txt", " текст"),
223         #(r"C:\Users\roman\PycharmProjects\algo_lab1\analysis\part_1_1mb.exe", " файл .exe"),
224         #("enwik7", "текстовый файл "),
225     ]
226
227     all_results = {}
228
229     for filename, description in test_files:
230         if not os.path.exists(filename):
231             print(f"\nфайл {filename} не найден, пропуск")
232             continue
233
234         with open(filename, 'rb') as f:
235             data = f.read()
236
237         results = test_all_compressors(data, filename, description)
238         all_results[filename] = results
239
240
241 if __name__ == "__main__":
242     main()
```